Forensic Analysis of Systemic Divergence in Dispatch Packet Generation: Causes, Mechanics, and Remediation Strategies

1. Executive Summary

This report constitutes a formal, expert-level forensic analysis of the "DP-MALFORMED" artifact generation failure currently observed within the Stela repository environment. The investigation was precipitated by an Operator query identifying a critical divergence between the historically validated output format (referenced herein as DP-SAMPLE) and the actual output (DP-MALFORMED) produced by the newly deployed TASK/DP system following the execution of the DP-PROMPT workflow.

The analysis establishes that the production of DP-MALFORMED does not represent a stochastic error or a failure of the generative model's logic capabilities. Rather, it is the deterministic result of a successful execution of a transitional instruction set that is structurally incongruent with the repository's legacy consumption logic. The system is currently in a "split-brain" state of governance evolution: the generative instructions (DP-PROMPT) have advanced to a "Phase 4" architecture utilizing intermediate data structures ("Slots Files"), while the validation infrastructure (tools/lint/dp.sh) and Operator expectations remain anchored in "Phase 3" document rendering standards.

The failure is rooted in the "Immutable Workflow Adoption" initiative (DP-OPS-0065), which introduced a canonical template system intended to eliminate manual structural edits. However, the tooling required to bridge the gap between the AI-generated intermediate data (the Slots File) and the final canonical surface (the Rendered DP) is either missing, undocumented, or unexecuted in the current workflow loop. Consequently, the AI creates a valid intermediate file that the system flags as malformed because it lacks the requisite Markdown headers enforced by the legacy linter.

This report provides a granular deconstruction of the Policy of Truth (PoT.md) to establish the governance context, reconstructs the precise logic path from prompt to failure, and offers three strategic remediation options. The primary recommendation—"Prompt-Side Rendering"—prioritizes immediate operational continuity by instructing the AI to emulate the behavior of the missing binary capability, thereby restoring the production of high-fidelity, compliant Dispatch Packets.

---

2. Constitutional Framework and Governance Physics

To accurately diagnose the "DP-MALFORMED" event, one must first rigorously map the "Physical Laws" of the environment as defined by the repository's constitution. In the Stela system, file outputs are not merely text; they are legal artifacts subject to strict jurisdictional and verification mandates. The failure of a file to parse is treated as a violation of these physical laws.

2.1 The Policy of Truth (PoT) Doctrine

The PoT.md file serves as the singular constitutional authority for the Stela environment.[1] Its preamble explicitly states: "The system prevents action that violates policy. It does not request compliance".[1] This distinction is vital for forensic analysis. It implies that the system's "System Failure States" (such as blocking a PR or stopping work) are functioning correctives, not bugs. The production of DP-MALFORMED triggered a "Stop Work" condition because it violated the structural expectations of the "Truth" predicate defined in the repository's axioms.

The analysis identifies three specific PoT Axioms that are in direct tension during this incident:

1. **SSOT (Single Source of Truth):** "One canonical file per domain; other mentions are pointers".[1] The transition to Phase 3/4 aims to make ops/src/surfaces/DP.md.tpl the SSOT for the Dispatch Packet structure.[1] Previously, the structure was copied manually into TASK.md, creating potential drift. The new workflow attempts to enforce the Template as SSOT by forbidding manual structural entry. DP-MALFORMED is an attempt to respect this by outputting only *content*, assuming the *structure* lives elsewhere (in the template).
2. **Drift:** "Any divergence between canon and repository state... is a failure state that requires stop and correction".[1] The TASK.md file is a Canon Surface.[1] If DP-MALFORMED is injected into TASK.md without being rendered into the proper headers, TASK.md ceases to conform to its schema. This constitutes "Canon Drift," triggering the system response: "**STOP WORK.** Reject PR".[1]
3. **Action:** "Deception or incoherence are inherently destabilizing and must be resolved".[1] The current state is incoherent: the prompt asks for one format (Slots), but the linter demands another (Rendered). This incoherence is the root cause of the "malformed" classification.

2.2 Filing Doctrine and Jurisdiction

The Filing Doctrine (PoT.md Section 1.1) segregates the repository into functional domains: ops/ (Run), docs/ (Explain), projects/ (Work), and storage/ (Trash).[1]

The DP-MALFORMED artifact appears to be an intermediate data object. Under strict Filing Doctrine, such an object belongs in storage/ (specifically storage/dp/intake/ or storage/tmp/) because it is not a final canonical artifact. However, the workflow effectively attempts to treat this intermediate object as a Canon Surface component by targeting it for TASK.md (active

work surface).

The jurisdiction of PoT.md is absolute across ops/, docs/, and projects/.[1] There are no parallel jurisdictions. Therefore, the AI worker cannot claim "I followed the prompt" as a defense if the prompt violates the higher-order PoT requirement for TASK.md integrity. The conflict resolution mechanism in PoT Section 1.2 states: "PoT is final authority; if conflict exists, stop and ask".[1] The generation of a malformed file is the system's way of forcing this "stop and ask" event.

2.3 Staffing Protocol and Logic Standards

The "Staffing Protocol" defines the Operator (Human) as the owner of final decisions and the Integrator (Lead AI) as the maintainer of governance.[1] The "Contractor" (Guest AI) executes logic.

In this incident, the Contractor executed the DP-PROMPT logic with "Absolute literalism" (a Behavioral Logic Standard).[1] The prompt requested a "Slots File," and the Contractor provided exactly that. The failure lies in the *Integrator's* failure to align the prompt's output specification with the Operator's validation tools.

Furthermore, the "Operational Directives" mandate "Anti-drift governance".[1] By generating a file that fundamentally alters the schema of TASK.md (replacing H2 headers with bracketed slots), the Contractor inadvertently introduced massive schema drift, triggering the automated enforcement gates.

---

3. Forensic Analysis of the Artifacts

To understand the mechanics of the failure, we must perform a comparative anatomical analysis of the three key artifacts: the target (DP-SAMPLE), the instruction (DP-PROMPT), and the anomaly (DP-MALFORMED).

3.1 DP-SAMPLE: The Legacy Target State

DP-SAMPLE.md represents the verified, compliant state. It is a document that passes all repository gates, specifically tools/lint/dp.sh.

Structural Anatomy of DP-SAMPLE:

The document is characterized by a nested Markdown hierarchy that serves both human readability and machine parsing (regex/grep).

Section	Content	Logic Purpose

Header	### DP-SAMPLE: Immutable workflow...	Identifies the DP transaction ID.
3.1 Freshness Gate	Base Branch: main, Base HEAD:...	Establishes the git baseline. Enforced by ops/bin/open checks.
3.1.1 Preflight	bash tools/lint/dp.sh...	Defines the validation gauntlet before edits.
3.2 Context Load	Lists PoT.md, SoP.md, etc.	Ensures the worker has the correct read-in order.
3.3 Scope	Objective:, In scope:, Out of scope:	Defines the boundaries of the transaction.
3.4 Execution Plan	3.4.1 State, 3.4.2 Request...	The narrative plan of action.
3.4.5 Receipt	List of commands to run.	The proof of work generation.
3.5 Closeout	Protocol and Closing Block.	The transaction commit/audit log.

Validation Signature: The tools/lint/dp.sh script (referenced in repo-gates.yml [1]) likely validates this file by grep-ing for these specific H2/H3 headers. If Section 3.3 Scope and Safety is missing, the lint fails.

3.2 DP-PROMPT: The Transitional Instruction

DP-PROMPT.md is the "program" running on the Contractor. A close reading of its content [1] reveals it is *not* a prompt to generate DP-SAMPLE. It is a prompt to generate a data structure for a specific binary: ops/bin/draft.

Instructional Constraints:

- **Format:** "Structure: **ops/bin/draft Slots File format**".[1]
- **Output Constraint:** "Output must be a valid Slots File compatible with ops/bin/draft".[1]
- **Formatting:** "Output only the stance format".[1]

The Mandatory Headers:

The prompt explicitly mandates a divergent header structure:

- `` (vs 3.2.2 DP-scoped load order)
- `` (vs Objective:)
- `` (vs In scope:)
- `` (vs 3.4.4 Patch / Diff)
- `` (vs 3.4.5 Receipt).[1]

The Logic Gap:

The prompt is asking for *data* (the filling), while TASK.md requires *document* (the pie). The prompt assumes the existence of a post-processing step (the ops/bin/draft tool) that will take this "Slots File" and render it into the DP-SAMPLE format using a template.

3.3 DP-MALFORMED: The Correctly Executed Error

DP-MALFORMED is the direct output of DP-PROMPT. It is "malformed" only in the context of the *legacy* TASK.md container. In the context of the ops/bin/draft binary, it is likely a perfectly valid input file.

Characteristics of DP-MALFORMED:

- **Missing Hierarchy:** It lacks the ## 3.x numbering scheme required by TASK.md Section 3.
- **Bracketed Headers:** It uses `` notation instead of Markdown headers.
- **Missing Boilerplate:** It likely omits sections like 3.1 Freshness Gate entirely, operating on the assumption that the Template (DP.md.tpl) will provide them automatically.

The "Malformation" Logic:

When the Operator pastes DP-MALFORMED into TASK.md, the tools/lint/dp.sh script runs. This script expects ## 3.1 Freshness Gate. It finds ``.

- **Result:** FAIL: Missing required section 3.1.
- **System Response: STOP WORK**.

Thus, DP-MALFORMED is not a hallucination. It is a **Type Error**. The AI output a SlotsFile object, but the System expected a MarkdownDocument object.

---

4. Workflow Synthesis (DP-PROMPT - DP-MALFORMED)

To fix the system, we must map the flow of information and identify exactly where the

transformation logic is missing.

4.1 The Intended "Phase 4" Workflow

Based on the evidence in DP-SAMPLE regarding "Immutable Workflow Adoption" and the existence of ops/bin/draft [1], the *intended* workflow is a compile-time generation process:

1. **Input:** Operator runs ops/bin/open.
2. **Prompting:** Operator provides DP-PROMPT + OPEN state to the AI.
3. **Data Generation:** AI analyzes the state and generates the Slots File (DP-MALFORMED).
   - *Content:* Objective, Plan, Changelog, Receipts.
   - *Format:* `` blocks.
4. **Compilation (The Missing Step):** The Operator saves this output to storage/dp/intake/DP-0066.slots and runs:
   - ./ops/bin/draft --ingest storage/dp/intake/DP-0066.slots
5. **Rendering:** The draft binary reads the slots, loads ops/src/surfaces/DP.md.tpl, substitutes {{OBJECTIVE}} with the content of ``, and writes the final Markdown.
6. **Injection:** The rendered Markdown is injected into TASK.md.

4.2 The Current "Broken" Workflow

The current workflow skips Step 4 and 5:

1. **Input:** Operator runs ops/bin/open.
2. **Prompting:** Operator provides DP-PROMPT to AI.
3. **Data Generation:** AI generates DP-MALFORMED (Slots).
4. **Direct Injection:** The Operator (or automation) attempts to paste DP-MALFORMED directly into TASK.md.
5. **Validation Failure:** tools/lint/dp.sh scans TASK.md, sees invalid structure, and rejects the PR.

4.3 Root Cause Analysis

The root cause is a **synchronization failure between the Prompt Engineering and the Tooling Maturity**.

- **The Prompt (DP-PROMPT)** is ahead of the curve. It assumes the "Drafting Tool" (ops/bin/draft) is fully operational and capable of parsing the "Slots File" format.
- **The Tooling (ops/bin/draft)** appears to be a simple shell script using sed for template substitution.[1] It likely does not yet have a sophisticated parser to read a multi-line "Slots File" with bracketed headers.
- **The Verification (lint/dp.sh)** is still enforcing the "Rendered" format. It has not been updated to validate "Slots Files" as legitimate DPs.

This creates a deadlock: The AI provides data for a tool that cannot read it (or isn't being

used), and the linter rejects the data because it isn't a document.

---

5. Systemic Risk Assessment

This failure state presents specific risks to the Stela repository's integrity, categorized by the PoT failure states.

5.1 Canon Drift Risk (High)

The most immediate risk is **Canon Drift**. If an operator forces DP-MALFORMED into TASK.md to "just get it working," they break the semantic structure of the governance surface. TASK.md is relied upon by ops/bin/open to generate context for future sessions.[1] If TASK.md is malformed, ops/bin/open may fail to extract the "Current DP" context, leading to **Context Amnesia** in subsequent sessions.

5.2 Context Hazard Risk (Medium)

DP-MALFORMED may inadvertently contain content that violates Context Hazard rules (e.g., pasting raw code from library/ paths) because the strict formatting guides of DP-SAMPLE (which include safety boilerplates) are missing from the raw slots. The "Safety Invariants" are present in the *data*, but without the *template* surrounding them, they may not be visually obvious to a human reviewer.

5.3 Ambiguity Risk (Extreme)

PoT defines **Ambiguity** as "Instructions capable of multiple interpretations" [1] and mandates a **STOP**. DP-MALFORMED is inherently ambiguous to the current toolchain. tools/lint/dp.sh cannot interpret it. A human reader might struggle to map `` to the expected Section 3.4.4. This ambiguity destabilizes the "Truth" of the repository state.

---

6. Strategic Options for Remediation

To resolve the incoherence and restore the production of DP-SAMPLE-compliant artifacts, three strategic vectors are available. These range from "Fix the Tool" to "Fix the Prompt."

Option 1: Tool-Side Rendering (The "North Star" Path)

This option validates the current DP-PROMPT strategy and focuses on fixing the missing execution step (The Compiler).

- **Concept:** The AI is functioning correctly as a "Drafting Engine." The breakdown is the lack of a "Rendering Engine." We must build or enable the capability to convert

DP-MALFORMED into DP-SAMPLE.

- **Action:**
  1. Update ops/bin/draft to accept a --slots argument.
  2. Implement a parser in ops/bin/draft (using awk or sed) to extract content between `` tags in the Slots File.
  3. Inject this content into ops/src/surfaces/DP.md.tpl replacing placeholders like {{OBJECTIVE}}.
- **Pros:**
  - **Strict Immutability:** Ensures every DP has byte-perfect boilerplate structure (derived from the template).
  - **Automation:** Reduces human error in formatting headers.
  - **Scalability:** Allows changing the DP format globally by editing one template.
- **Cons:**
  - **High Effort:** Requires writing a robust parser in Bash (brittle/complex).
  - **Blocking:** Work cannot proceed until the binary is updated.
  - **Risk:** sed/awk parsers are prone to breaking on special characters or multi-line strings in AI output.

Option 2: Prompt-Side Rendering (The "Pragmatic" Path)

This option accepts that the tooling is not yet ready to ingest Slots Files and shifts the burden of rendering back to the AI worker.

- **Concept:** Instruct the AI to act as *both* the Drafter and the Renderer. Instead of outputting the intermediate Slots File, the AI will ingest the Template (provided in context) and output the fully rendered Markdown document.
- **Action:**
  1. Modify DP-PROMPT to remove the "Slots File" constraint.
  2. Inject the content of ops/src/surfaces/DP.md.tpl into the prompt.
  3. Instruct the AI: "Populate the attached template with the plan data. Output the valid Markdown."
- **Pros:**
  - **Immediate Fix:** No code changes required. Just a prompt update.
  - **Compatibility:** Produces DP-SAMPLE format immediately, passing existing lints.
  - **Human Readability:** The output is instantly verifiable by the Operator.
- **Cons:**
  - **Drift Risk:** AI might hallucinate slight changes to the boilerplate (e.g., changing "Freshness Gate" to "Freshness Check"), failing the strict structure lint.
  - **PoT Violation:** Technically violates the "Immutable Workflow" ideal of "No manual structural edits" if the AI is considered a manual editor.

Option 3: Schema Definition Update (The "Data-First" Path)

This option fundamentally changes the definition of a valid DP to accept the Slots File as the

new standard.

- **Concept:** TASK.md no longer holds the full narrative. It holds the data blocks. The "Document" metaphor is abandoned for a "Record" metaphor.
- **Action:**
  1. Update tools/lint/dp.sh to pass if it sees `` instead of ## 3.3 Scope.
  2. Update docs/MANUAL.md to reflect the new format.
- **Pros:**
  - **Machine Readable:** Easier for future agents to parse.
  - **Concise:** Removes repetitive boilerplate from the repo history.
- **Cons:**
  - **Massive Refactor:** Breaks compatibility with SoP history, PoW expectations, and human reading habits.
  - **Ambiguity:** "Malformation" becomes the new "Normal," confusing operators trained on legacy docs.

---

7. Recommendation and Proposed Solution

Recommendation: Adopt Option 2 (Prompt-Side Rendering) as the immediate tactical fix to restore repository velocity, while queueing **Option 1 (Tool-Side Rendering)** as a strategic technical debt item for the Platform Engineering roadmap.

The current "malformed" state is a user-experience failure caused by a prompt that is effectively "future-traveling" to a tooling state that does not yet exist. The repository lints (tools/lint/dp.sh) are strict and currently enforce the DP-SAMPLE structure. Fighting the lints to support an intermediate format is counter-productive.

7.1 Proposed Workflow: The "Hybrid" Prompt

The modified workflow keeps the "Immutable Surface" philosophy by providing the template to the AI, but asks the AI to perform the compilation step.

New Prompt Logic:

1. **Context:** "You are the Drafting Engine. Here is the Canonical Template (DP.md.tpl)."
2. **Task:** "Analyze the requirement. Fill the Template slots."
3. **Output:** "Output the **Rendered** Dispatch Packet. Do not output the intermediate slots."

This satisfies PoT.md because the **structure** still originates from the SSOT (the template provided in the prompt), even if the AI is the mechanism of combination.

7.2 Formulation: The Corrected DP-PROMPT

To produce a DP identical to DP-SAMPLE, the Operator should deploy the following logic in

the DP-PROMPT:

CONSTRUCT: DP-OPS-0066 (Full Render)

Directive:

Instead of a "Slots File," you must output the fully rendered Dispatch Packet.

The structure must be byte-compatible with ops/src/surfaces/DP.md.tpl.

Constraint:

- Use standard H2/H3 Markdown headers (e.g., ## 3.1 Freshness Gate).
- Do NOT use bracketed data headers (e.g., ``).
- Populate the following sections based on the template logic:

1. **Header:** ### DP-OPS-XXXX:
2. **Freshness Gate:** ## 3.1 Freshness Gate (Include exact git commands).
3. **Preflight:** ## 3.1.1 DP Preflight Gate (Include exact lint commands).
4. **Context:** ## 3.2 Required Context Load (Use canonical load order).
5. **Scope:** ## 3.3 Scope and Safety (Populate Objective, In/Out Scope).
6. **Plan:** ## 3.4 Execution Plan (Detail the Patch/Diff A-E).
7. **Receipts:** ## 3.4.5 Receipt (List verification commands).
8. **Closeout:** ## 3.5 Closeout (Include standard protocol).

Verification:

Your output must pass bash tools/lint/dp.sh TASK.md. This means all boilerplate headers must be present and exact.

7.3 Conclusion

The DP-MALFORMED artifact is a symptom of a **Partial Migration**. The logic (Prompt) migrated to Phase 4, while the body (Repository) remained in Phase 3. By realigning the Prompt to respect the current constraints of the Repository's physical laws (tools/lint/dp.sh), the system will return to equilibrium, producing DP-SAMPLE compliant artifacts that pass all gates and advance the SoP.md ledger.

---

8. Strategic Options Menu

Option	Name	Action	Pros	Cons	Risk
1	**Prompt-Side Rendering**	**Rewrite DP-PROMPT.md** to request full Markdown structure matching DP-SAMPLE and DP.md.tpl.	Restores immediate productivity; passes existing lints; readable by humans.	AI may drift from template structure; relies on AI discipline rather than code enforcement.	Low (Corrections are easy via review).
2	**Tool-Side Rendering**	**Update ops/bin/draft** to parse DP-MALFORMED and render it into TASK.md.	Enforces perfect template compliance; strictly follows Phase 4 "Immutable" doctrine.	Requires immediate coding work on ops/bin/draft; adds friction step for Operator.	Medium (Tooling bugs could block work).
3	**Schema Redefinition**	**Update tools/lint/dp.sh** to accept the "Slots File" format as valid.	Validates the current output as "Correct"; moves towards machine-readable definition files.	Breaks compatibility with TASK.md narrative; confusing for human operators expecting docs.	High (Governance drift).

Recommendation: Option 1. Align the prompt to the reality of the current linting infrastructure to resolve the "Stop Work" condition immediately.

Works cited

1. dump-full-main-b7a7a2eb.txt